

Assignment 1: Observability

QuickLink — Instrumented URL Shortener

Enterprise Software Development — Fall 2026
Machine / Node label: **Abdullah Naveed**

September 23, 2026

Contents

1	Part A — Project	2
1.1	Problem statement	2
1.2	Intended users	2
1.3	Solution	2
1.4	What currently works	2
1.5	How to run / try it	2
2	Part B — Metrics	2
2.1	Infrastructure	2
2.2	Four metric types	3
2.3	Metrics table	3
2.4	Grafana charts and PromQL	3
3	Part C — Logs	4
3.1	Pipeline	4
3.2	What is logged (and what is not)	4
3.3	Example raw log line	4
3.4	Searching in Kibana	4
3.5	Survival across restarts	4
4	Part D — System Design	4
4.1	Architecture diagram	4
4.2	Failure modes	5
4.3	Trace one metric (links_created_total)	5
4.4	Trace one log (request handled)	5
5	Part E — Experiments	5
5.1	E1 — Fault injection (slow redirects)	5
5.2	E2 — Cardinality explosion	6

1 Part A — Project

1.1 Problem statement

People routinely share long, fragile URLs (documentation, tickets, demo endpoints). They need a short, reliable redirect they can paste into chat, slides, or terminals without copying an unwieldy string.

1.2 Intended users

Developers and students who need disposable short links during demos, labs, and debugging—not a production multi-tenant SaaS.

1.3 Solution

QuickLink is a small FastAPI URL shortener:

- `POST /shorten?url=...` creates a 6-character code stored in SQLite (plus an in-memory cache).
- `GET /{code}` returns HTTP 307 to the original URL.
- `GET /metrics` exposes Prometheus metrics.
- Structured JSON logs go to stdout and are shipped Filebeat → Elasticsearch → Kibana.

The stack is orchestrated with Docker Compose (app, Prometheus, Grafana, Node Exporter, Elasticsearch, Kibana, Filebeat).

1.4 What currently works

End-to-end shorten and redirect; Prometheus scrape of app and Node Exporter; Grafana dashboard (application + host CPU/memory/disk/network); ELK log pipeline; reversible fault injection (`FAULT_INJECT`); cardinality experiment (`CARDINALITY_EXPERIMENT` + `POST /demo/hit`).

1.5 How to run / try it

```
cd ABD_HW
docker compose up -d
curl.exe -X POST "http://localhost:8000/shorten?url=https://example.com"
curl.exe -i "http://localhost:8000/<code>"
```

Useful URLs: app docs <http://localhost:8000/docs>; Grafana <http://localhost:3000> (admin/admin); Prometheus <http://localhost:9090>; Kibana <http://localhost:5601>.

Teardown: `docker compose down` (keep volumes) or `docker compose down -v` (wipe volumes including ES data).

2 Part B — Metrics

2.1 Infrastructure

- **Prometheus** scrapes `app:8000/metrics` and `node-exporter:9100` every **5 s** (`prometheus.yml`).
- **Grafana** is provisioned with datasource `http://prometheus:9090` and dashboard *QuickLink Observability* (`quicklink-obs`).
- **Node Exporter** panels cover CPU, memory, disk, and network.
- The measured instance is labeled **Abdullah Naveed** in `prometheus.yml`. On Docker Desktop (Windows), Node Exporter sees the Linux VM / container host, not bare-metal Windows—stated here for viva clarity.

2.2 Four metric types

All four Prometheus metric types are used across the project (not per category): Counter, Gauge, Histogram, and Summary.

Python Summary note (required). The Prometheus Python client *Summary* does **not** provide reliable server-side percentiles across scrapes the way Histograms do. Summaries are used for **average** lookup time only. Percentiles (p95) come from the Histogram via `histogram_quantile()`.

2.3 Metrics table

Metric	Type	Purpose	Unit	Labels	Where
links_created_total	Counter	Business: links created	count	none	short
links_stored_total	Gauge	Business: DB row count (self-explored)	count	none	short start
http_requests_failed_total	Counter	App: failures / 404s	count	endpoint	middle
requests_in_progress	Gauge	App: in-flight load	count	none	middle
http_request_duration_seconds	Histogram	App: latency buckets for p95	seconds	endpoint	middle
redirect_lookup_duration_seconds	Summary	App: avg resolve time	seconds	none	[0.1, redirect]
demo_requests_total	Counter	E2 cardinality demo	count	request_id iff experiment on	demo IDs

Table 1: Application and experiment metrics.

2.4 Grafana charts and PromQL

Chart	PromQL	Shows
Links created rate	<code>rate(links_created_total[1m])</code>	Creates/sec
Links stored	<code>links_stored_total</code>	Current DB size
In progress	<code>requests_in_progress</code>	Concurrent handlers
HTTP p95	<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m]) by (le)))</code>	95th % latency; 5 m rate
Lookup avg	<code>rate(sum[5m])/rate(count[5m])</code> on Summary	Mean resolve time
Failed rate	<code>rate(http_requests_failed_total[5m])</code>	Failures/sec
Node CPU %	<code>100 - (avg by(instance)(irate(...idle...[5m]))*100)</code>	Abdullah Naveed
Node memory	<code>node_memory_MemAvailable.bytes</code>	Available RAM
Node disk used %	<code>100 - (avail/size)*100</code> on mountpoint /	Root FS used
Node network	<code>rate(node_network_*_bytes_total{device!="lo"}[5m])</code>	Network throughput

Table 2: Dashboard queries (file: `grafana/provisioning/dashboards/json/quicklink.json`).

Self-explored metric. `links_stored_total` tracks dataset size over time (not just request traffic), which class request-level examples alone do not cover.

3 Part C — Logs

3.1 Pipeline

App (JSON stdout) → Docker json-file → Filebeat → Elasticsearch → Kibana
Discover

Filebeat reads `/var/lib/docker/containers/*/*.log`, decodes the nested JSON message into fields under `ql.*` (to avoid clashing with reserved Beat/ES fields), and indexes `quicklink-logs-%{+yyyy.MM.dd}` (data stream `.ds-quicklink-logs-*`). ILM is **disabled**; retention is manual (`docker compose down -v` or delete indices).

3.2 What is logged (and what is not)

JSON fields include: `timestamp`, `time`, `service=quicklink`, `severity`, `message`, and `request_id` (middleware). Link creation logs code and **URL host only** (no query strings / tokens). **Never** log secrets, passwords, Authorization headers, or personal data.

Implementation: `log_json()` in `app.py`; called from middleware, `shorten()`, and `redirect()`.

3.3 Example raw log line

```
{"timestamp":"2026-09-23T13:20:59.010718+00:00","time":1790169659.01,
"service":"quicklink","severity":"INFO","message":"request handled",
"request_id":"60acc223-...", "path":"/shorten", "status":200, "duration":0.012}
```

3.4 Searching in Kibana

1. Open <http://localhost:5601> → Discover.
2. Data view: `quicklink-logs-*`; time field `@timestamp`.
3. Example queries: `message: "link created"`; `message: "not found"`; `ql.request_id: "<uuid>"` (or `request_id` depending on field mapping).

3.5 Survival across restarts

App process logs are ephemeral unless shipped. Elasticsearch holds indexed documents across container restarts *until* volumes are removed. Filebeat registry state survives in its container volume. Prometheus TSDB persists similarly until wiped.

4 Part D — System Design

4.1 Architecture diagram

Communication / storage. Users hit HTTP on port 8000. Prometheus pulls metrics over HTTP. Grafana queries Prometheus. Filebeat pushes logs to Elasticsearch over HTTP. SQLite keeps link mappings local and simple for Part A; metrics and logs are deliberately separated so dashboard load does not contend with log search.

What I do not fully understand yet. Exact Docker Desktop WSL2 cgroup accounting behind Node Exporter numbers on Windows, and every Elasticsearch mapping edge case when Beat fields collide with app JSON (mitigated by decoding into `ql.*`).

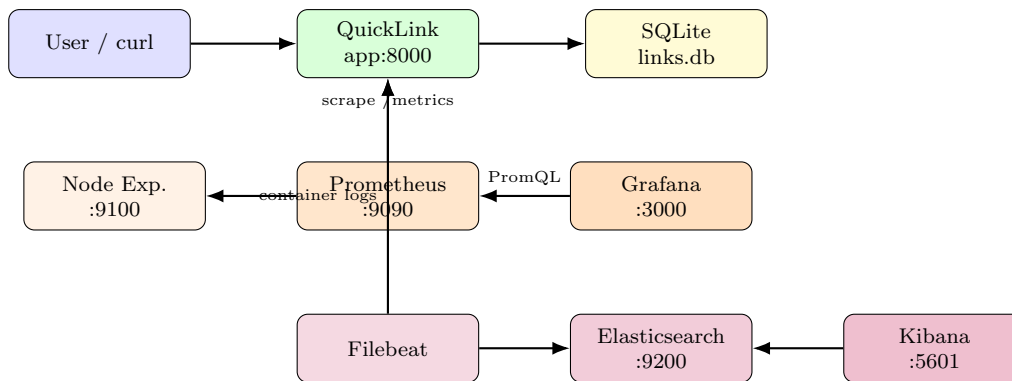


Figure 1: QuickLink observability architecture (Compose network; service DNS names).

Component	If it fails	User impact
App	Connection errors	Direct outage
Prometheus	Scrapes stop; TSDB stale	Users unaffected; dashboards freeze
Grafana	UI down	Observability UI only
Node Exporter	Host panels empty	App still works
Elasticsearch	Filebeat buffers/retries	Users unaffected; logs delayed
Filebeat	Logs stop shipping	Users unaffected
Kibana	Search UI down	Data still in ES

Table 3: Component failure behaviour.

4.2 Failure modes

4.3 Trace one metric (links_created_total)

1. **Code:** `shorten()` calls `links_created_total.inc()`.
2. **Expose:** `GET /metrics` via `generate_latest()`.
3. **Prometheus:** job `quicklink` scrapes every 5s; store time series.
4. **Grafana:** panel query `rate(links_created_total[1m])`.

4.4 Trace one log (request handled)

1. **Code:** `middleware log_json("INFO", "request handled", ...)`.
2. **Docker:** wraps stdout in json-file log.
3. **Filebeat:** reads container log; `decode_json_fields → ql.*`.
4. **Elasticsearch:** document in `.ds-quicklink-logs-*`.
5. **Kibana:** Discover search message: `"request handled"`.

5 Part E — Experiments

5.1 E1 — Fault injection (slow redirects)

Mechanism. With `FAULT_INJECT=1`, every 5th redirect executes `time.sleep(0.5)` in `redirect()` (`app.py`). Reversible by setting `FAULT_INJECT=0` and recreating the app container.

Prediction (written before the fault run). p95 of `http_request_duration_seconds` and the Summary lookup average will rise; INFO logs will show larger duration on every 5th redirect; failure counters should stay flat (latency, not errors).

Commands.

```
$code = (curl.exe -s -X POST "http://localhost:8000/shorten?url=https://example.com" '
| ConvertFrom-Json).code
1..80 | ForEach-Object { curl.exe -s -o NUL "http://localhost:8000/$code" }
curl.exe -s -G "http://localhost:9090/api/v1/query" --data-urlencode '
"query=histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[2m])) by (le))"
# Toggle FAULT_INJECT in docker-compose.yml, then:
docker compose up -d --force-recreate app
```

Scrape interval is 5s; each phase ran long enough for multiple scrapes. Grafana time range: last 15–30 minutes covering each phase.

Phase	p95 latency
Baseline (FAULT_INJECT=0)	≈ 0.095 s
Fault on (FAULT_INJECT=1)	≈ 0.86 s
Recovery (FAULT_INJECT=0, shortly after)	falling toward baseline (1 m rate window still overlaps fault sample)

Table 4: Anomaly experiment — Histogram p95.

Measured results (this machine).

Root cause and user impact. Injected sleep adds ~ 500 ms to every 5th redirect ($\sim 20\%$ of follows). Users experience intermittent slow link opens without HTTP errors.

Recovery. Fault flag returned to 0; app recreated; load repeated. After the Prometheus rate window rolls past the fault interval, p95 returns near the baseline. Fault is fully reversible.

5.2 E2 — Cardinality explosion

Setup. Temporary counter `demo_requests_total` with a `request_id` label when `CARDINALITY_EXPERIMENT=1`. Unique IDs are **capped at 100** in code. Endpoint: `POST /demo/hit`.

Mode	count(demo_requests_total) after 100 hits
High cardinality (<code>request_id</code> label)	100 series
Low cardinality (no label)	1 series

Table 5: Cardinality experiment.

Results.

Interpretation. Each unique label value creates a new time series $\rightarrow$ RAM growth and slower queries at scale. High-cardinality identifiers (request IDs) belong in **logs**, not metric labels. Removing the label / restarting does **not** instantly delete historical series already stored in Prometheus.

Prometheus was *not* crashed; the demo stayed within the 100-ID cap.